Argo CD — 3–4 YOE Scenario-Based Q&A

1. Argo CD application is showing OutOfSync. What will you check?
Ans: First, I check the Argo CD application status and identify which Kubernetes resources are showing differences. Then I compare the Git manifests with the resources currently running in the cluster.
I also check whether someone manually changed the Kubernetes resource or whether the Git manifest was recently modified. After identifying the difference, I decide whether to update Git or sync the application.
Simple ga gurthupettuko: Application OutOfSync lo unte → first difference ye resource lo undo chudu → Git manifest and cluster resource compare cheyyi → manual change jariginda check cheyyi → Git change ayyinda chudu → required correction chesi application sync cheyyi.
Eng: If the Argo CD application is OutOfSync → first identify which resource is different → compare the Git manifest with the cluster resource → check for manual changes → check recent Git changes → correct the configuration and sync the application.
Example:
argocd app get myapp → application OutOfSync lo unda chudu.
argocd app diff myapp → Git and cluster madhya exact difference chudu.
kubectl get deployment myapp -o yaml → cluster lo actual configuration check cheyyi.
Git manifest tho compare chesi → required change Git lo cheyyi → argocd app sync myapp → application Synced ayyinda verify cheyyi.

---

2. Argo CD application sync is failing. What will you check?
Ans: First, I check the Argo CD application events and sync status to identify which resource is failing during synchronization. Then I check the Kubernetes resource events and error messages.
I also verify the manifest configuration, namespaces, permissions, and required dependencies. After fixing the actual Kubernetes or manifest issue, I run the sync again and verify the application health.
Simple ga gurthupettuko: Sync fail ayithe → first sync status and events chudu → ye resource fail ayindo identify cheyyi → Kubernetes events and error check cheyyi → manifest and namespace verify cheyyi → permissions and dependencies check cheyyi → issue fix chesi sync malli run cheyyi.
Eng: If the Argo CD sync fails → first check sync status and events → identify the failed resource → check Kubernetes events and errors → verify manifests and namespace → check permissions and dependencies → fix the issue and run sync again.
Example:
argocd app sync myapp → sync fail ayyindo chudu.
argocd app get myapp → failed resource identify cheyyi.
kubectl describe deployment myapp -n prod → Kubernetes error exact ga chudu.
kubectl get events -n prod --sort-by=.lastTimestamp → recent events check cheyyi.
Issue fix chesi → argocd app sync myapp → sync successful ayyinda verify cheyyi.

---

3. Argo CD application is showing Health: Degraded. What will you check?
Ans: First, I check which resource is causing the application health to become Degraded. Then I inspect the Pod, Deployment, Service, or other affected resource and check its Kubernetes events and logs.
I also verify readiness probes, resource configuration, image availability, and application health. After fixing the unhealthy resource, I verify that Argo CD reports the application as Healthy.
Simple ga gurthupettuko: Health Degraded unte → first ye resource unhealthy ga undo chudu → aa resource status check cheyyi → events and logs verify cheyyi → probes, image and resources check cheyyi → root cause fix chesi Argo CD health verify cheyyi.
Eng: If the Argo CD application is Degraded → first identify the unhealthy resource → check its status → verify events and logs → check probes, image, and resources → fix the root cause and verify the Argo CD health status.
Example:
argocd app get myapp → Health: Degraded ye resource valla vachindo chudu.
kubectl get pods -n prod → affected Pod status check cheyyi.
kubectl describe pod <pod-name> -n prod → events lo error chudu.
kubectl logs <pod-name> -n prod → application error unda verify cheyyi.
Issue fix chesi → argocd app get myapp → Healthy ayyinda verify cheyyi.

---

4. Argo CD Auto-Sync is not working. What will you check?
Ans: First, I check whether automated sync is enabled for the Argo CD application. Then I verify whether Argo CD can access the Git repository and detect the latest commit.
I also check whether the application is already Synced, whether sync policies contain restrictions, and whether there are errors in the application events. After fixing the configuration or repository issue, I verify that a new Git change triggers synchronization.
Simple ga gurthupettuko: Auto-Sync work avvakapothe → first automated sync enabled unda chudu → Git latest commit detect chesthunda check cheyyi → repository access verify cheyyi → sync policy and application events check cheyyi → issue fix chesi new Git change tho auto-sync verify cheyyi.
Eng: If Argo CD Auto-Sync is not working → first check whether automated sync is enabled → verify the latest Git commit is detected → check repository access → verify sync policy and application events → fix the issue and confirm Auto-Sync with a new Git change.
Example:
argocd app get myapp → Sync Policy lo automated sync enabled unda check cheyyi.
argocd app get myapp → application latest Git commit tho compare avutunda chudu.
argocd repo list → Git repository connection successful unda verify cheyyi.
Git lo small configuration change push chesi → Argo CD application status chudu → Auto-Sync trigger ayyinda verify cheyyi.

5. Argo CD cannot access the Git repository. What will you check?
Ans: First, I check the repository connection status in Argo CD and identify whether the issue is related to authentication, URL, or network connectivity. Then I verify the repository URL and credentials.
If the repository is private, I check whether the configured SSH key or access token is valid and has the required permissions. After correcting the repository configuration, I test the connection again.
Simple ga gurthupettuko: Argo CD Git repository access avvakapothe → first repository status chudu → URL and authentication check cheyyi → private repo ayithe SSH key or token verify cheyyi → permissions and network check cheyyi → configuration fix chesi connection test cheyyi.
Eng: If Argo CD cannot access the Git repository → first check the repository status → verify the URL and authentication → check the SSH key or token for a private repository → verify permissions and network → fix the configuration and test the connection.
Example:
argocd repo list → repository connection Successful ga unda chudu.
argocd repo get <repo-url> → repository configuration check cheyyi.
Private repository ayithe → SSH key or token valid ga unda verify cheyyi.
Git repository access test chesi → argocd repo list → connection successful ayyinda verify cheyyi.

---

6. Argo CD is deploying the wrong Kubernetes manifest. What will you check?
Ans: First, I check the Git repository, branch, and path configured in the Argo CD application. Then I verify whether the application is pointing to the expected environment and manifest directory.
I also check the commit revision currently being deployed and compare the generated manifest with the Git source. After correcting the repository, branch, path, or revision, I sync the application again.
Simple ga gurthupettuko: Wrong manifest deploy ayithe → first repo, branch and path chudu → correct environment aa check cheyyi → deployed revision verify cheyyi → Git manifest tho compare cheyyi → wrong configuration correct chesi sync malli run cheyyi.
Eng: If Argo CD deploys the wrong manifest → first check the repository, branch, and path → verify the correct environment → check the deployed revision → compare with the Git manifest → correct the configuration and sync again.
Example:
argocd app get myapp → Repo, Target Revision, Path check cheyyi.
git branch → correct branch use chesthunnara verify cheyyi.
git log -1 → latest expected commit unda chudu.
Argo CD application configuration correct chesi → argocd app sync myapp → expected manifest deploy ayyinda verify cheyyi.

---

7. Argo CD shows Drift in a Kubernetes resource. What will you check?
Ans: First, I identify which resource has drifted and compare the live Kubernetes resource with the desired Git configuration. Then I check whether someone manually changed the resource in the cluster.
I also verify whether the difference is expected or caused by a Kubernetes controller modifying the resource automatically. Based on the cause, I either update Git or allow Argo CD to restore the desired configuration.
Simple ga gurthupettuko: Drift vaste → first ye resource different ga undo chudu → Git and live resource compare cheyyi → manual change jariginda check cheyyi → Kubernetes controller change chesinda verify cheyyi → required ayithe Git update cheyyi or Argo CD tho desired state restore cheyyi.
Eng: If Argo CD detects drift → first identify the different resource → compare Git with the live resource → check for manual changes → verify whether a Kubernetes controller changed it → update Git if required or restore the desired state through Argo CD.
Example:
argocd app diff myapp → exact drift ekkada undo chudu.
kubectl get deployment myapp -o yaml → live configuration check cheyyi.
Git manifest tho compare chesi → manual change aa controller change aa identify cheyyi.
Required configuration Git lo correct chesi → argocd app sync myapp → drift remove ayyinda verify cheyyi.

---

8. Argo CD application is stuck in Syncing state. What will you check?
Ans: First, I check the application status and identify which resource is taking time or failing during synchronization. Then I check the resource events, Pod status, and Argo CD application events.
I also verify whether there is a dependency issue, hook waiting for completion, or resource that cannot become Ready. After identifying the blocking resource, I fix the issue and monitor the synchronization again.
Simple ga gurthupettuko: Syncing lo stuck ayithe → first ye resource daggara stuck undo chudu → application and resource events check cheyyi → Pod status verify cheyyi → dependency or hook issue unda chudu → blocking resource fix chesi sync status malli verify cheyyi.
Eng: If the Argo CD application is stuck in Syncing → first identify the resource causing the delay → check application and resource events → verify Pod status → check dependency or hook issues → fix the blocking resource and verify the sync status again.
Example:
argocd app get myapp → Syncing lo ye resource stuck undo chudu.
kubectl get pods -n prod → Pod Pending or failing ga unda check cheyyi.
kubectl get events -n prod --sort-by=.lastTimestamp → blocking error identify cheyyi.
Issue fix chesi → argocd app get myapp → Synced ayyinda verify cheyyi.

9. Argo CD rollback is required after a bad deployment. What will you do?
Ans: First, I confirm that the latest deployment caused the application issue and identify the last known good revision. Then I review the Argo CD application history to identify the stable deployment.
I rollback to the previous stable revision and monitor the Kubernetes resources. After rollback, I verify the application health, Pod status, and Service functionality.
Simple ga gurthupettuko: Bad deployment ayithe → first latest version problem aa confirm cheyyi → last stable revision identify cheyyi → Argo CD history check cheyyi → previous stable version ki rollback cheyyi → Pods and application health verify cheyyi → Service working unda check cheyyi.
Eng: If a bad deployment needs rollback → first confirm the latest version caused the issue → identify the last stable revision → check Argo CD history → rollback to the stable version → verify Pods and application health → check the Service.
Example:
argocd app history myapp → previous stable revision identify cheyyi.
argocd app get myapp → current bad revision confirm cheyyi.
argocd app rollback myapp <revision> → required stable revision ki rollback cheyyi.
kubectl get pods -n prod → new Pods healthy ga unnaya chudu.
argocd app get myapp → application Healthy and Synced ayyinda verify cheyyi.

---

10. Argo CD deployment is failing because of Kubernetes permissions. What will you check?
Ans: First, I check the Argo CD application error and identify which Kubernetes resource operation is being denied. Then I verify the ServiceAccount, Role, ClusterRole, and RoleBinding permissions used by Argo CD.
I also check whether the resource is being created in the correct namespace and whether Argo CD has permission to manage that resource type. After correcting the RBAC configuration, I sync the application again and verify the deployment.
Simple ga gurthupettuko: Permission valla deployment fail ayithe → first exact permission error chudu → Argo CD ServiceAccount identify cheyyi → Role or ClusterRole check cheyyi → RoleBinding verify cheyyi → correct namespace and resource permission chudu → RBAC fix chesi sync malli run cheyyi.
Eng: If the Argo CD deployment fails because of permissions → first check the exact permission error → identify the Argo CD ServiceAccount → verify Role or ClusterRole → check RoleBinding → verify namespace and resource permissions → fix RBAC and run sync again.
Example:
argocd app get myapp → permission error exact ga chudu.
kubectl get serviceaccount -n argocd → Argo CD ServiceAccount identify cheyyi.
kubectl get role,rolebinding -n argocd → namespace permissions check cheyyi.
kubectl get clusterrole,clusterrolebinding → cluster-level permissions verify cheyyi.
RBAC permission correct chesi → argocd app sync myapp → deployment successful ayyinda verify cheyyi.

---

11. Argo CD application is stuck in Progressing state. What will you check?
Ans: First, I check the Application status and identify which Kubernetes resource is still progressing. Then I check the resource events and Pod status to understand whether the deployment is waiting for a Pod, Service, or another resource to become healthy.
I also verify whether there are image, scheduling, readiness probe, or configuration issues. After fixing the actual Kubernetes issue, I refresh the Argo CD application and verify that it becomes Healthy and Synced.
Simple ga gurthupettuko: Application Progressing lo unte → first ye resource stuck ayindo chudu → resource events and Pod status check cheyyi → image, scheduling and readiness issues verify cheyyi → issue fix cheyyi → Argo CD refresh chesi Healthy ayyinda verify cheyyi.
Eng: If the Argo CD application is stuck in Progressing → first identify the resource causing the delay → check events and Pod status → verify image, scheduling and readiness issues → fix the issue → refresh Argo CD and verify Healthy status.
Example: kubectl get pods -n app → Pod Pending or NotReady unda chudu.
kubectl describe pod <pod-name> -n app → exact scheduling or readiness error chudu.
kubectl logs <pod-name> -n app → application error unda check cheyyi.
Issue fix chesi → Argo CD application refresh → **Healthy** ayyinda verify cheyyi.

---

12. Argo CD application is OutOfSync. What will you check?
Ans: First, I check the differences between the desired state in Git and the live state in the Kubernetes cluster. Then I identify which resources or fields are different.
I verify whether the changes were made manually in the cluster or whether the Git configuration was recently updated. Based on the requirement, I either sync the application or correct the Git configuration.
Simple ga gurthupettuko: Application OutOfSync lo unte → first Git and cluster difference chudu → ye resource change ayindo identify cheyyi → manual change aa Git change aa check cheyyi → correct state decide cheyyi → required ayithe Sync cheyyi.
Eng: If the Argo CD application is OutOfSync → first compare Git and live state → identify the changed resource → check whether the change was manual or from Git → decide the correct state → sync the application if required.
Example: Argo CD lo **OutOfSync** → Application open chesi **Diff** chudu.
kubectl get deployment <name> -o yaml → live configuration check cheyyi.
Git manifest compare cheyyi → difference ekkada undo identify cheyyi.
Correct state Git lo unte → **Sync** run chesi → application **Synced** ayyinda verify cheyyi.

13. Argo CD sync is failing. What will you check?
Ans: First, I check the Argo CD sync operation details and identify which resource failed during synchronization. Then I check the Kubernetes events and resource configuration for the failed resource.
I also verify RBAC permissions, invalid manifests, missing dependencies, namespace issues, and resource conflicts. After fixing the root cause, I run the sync again and verify the application status.
Simple ga gurthupettuko: Sync fail ayithe → first ye resource daggara fail ayindo chudu → sync error check cheyyi → Kubernetes events and manifest verify cheyyi → permissions, namespace and resource conflict check cheyyi → issue fix chesi Sync malli run cheyyi.
Eng: If the Argo CD sync fails → first identify the failed resource → check the sync error → verify Kubernetes events and the manifest → check permissions, namespace, and resource conflicts → fix the issue and sync again.
Example: Argo CD lo **Sync Failed** → failed resource identify cheyyi.
kubectl describe <resource> <name> -n <namespace> → exact error chudu.
kubectl get events -n <namespace> --sort-by=.lastTimestamp → Kubernetes event check cheyyi.
Permission issue ayithe RBAC fix cheyyi → manifest issue ayithe Git lo fix cheyyi → **Sync** malli run cheyyi.

---

14. Argo CD is unable to connect to the Kubernetes cluster. What will you check?
Ans: First, I check the Argo CD cluster connection status and identify whether the problem is authentication, network connectivity, or cluster configuration. Then I verify the cluster endpoint and credentials configured in Argo CD.
I also check whether the Kubernetes API server is reachable and whether the Argo CD components have the required permissions. After fixing the connection issue, I verify the cluster status in Argo CD.
Simple ga gurthupettuko: Argo CD cluster connect avvakapothe → first cluster connection status chudu → API endpoint and credentials verify cheyyi → network access check cheyyi → Argo CD permissions verify cheyyi → issue fix chesi cluster connection verify cheyyi.
Eng: If Argo CD cannot connect to the Kubernetes cluster → first check cluster connection status → verify API endpoint and credentials → check network access → verify Argo CD permissions → fix the issue and verify the connection.
Example: argocd cluster list → cluster **Successful** or connection error unda chudu.
kubectl cluster-info → Kubernetes API reachable unda check cheyyi.
Argo CD cluster credentials and endpoint verify cheyyi → network or RBAC issue fix cheyyi → argocd cluster list malli run cheyyi.

---

15. Argo CD repository connection is failing. What will you check?
Ans: First, I check the repository connection status and identify whether the issue is related to the repository URL, authentication, network, or certificate. Then I verify the configured repository credentials and access permissions.
I also check whether Argo CD can reach the Git repository and whether the repository URL is correct. After correcting the issue, I test the repository connection again.
Simple ga gurthupettuko: Git repo connect avvakapothe → first repo status chudu → repository URL and credentials verify cheyyi → Git access and network check cheyyi → certificate issue unda chudu → issue fix chesi repo connection test cheyyi.
Eng: If Argo CD cannot connect to the Git repository → first check repository status → verify the repository URL and credentials → check Git access and network → verify certificate issues → fix the problem and test the connection again.
Example: Argo CD **Settings → Repositories** → repository status chudu.
Repository URL correct aa check cheyyi.
Git credentials verify cheyyi → git ls-remote <repo-url> → repository access avutunda check cheyyi.
Authentication or certificate issue fix chesi → repository connection **Successful** ayyinda verify cheyyi.

---

16. Argo CD is not detecting the latest Git commit. What will you check?
Ans: First, I verify whether the latest commit is actually present in the configured Git branch. Then I check whether Argo CD is monitoring the correct repository, branch, and path.
I also verify the repository connection and refresh the application to trigger a new comparison. If automatic refresh is configured, I check whether there is any repository polling or webhook issue.
Simple ga gurthupettuko: Latest Git commit detect avvakapothe → first commit branch lo unda chudu → repo, branch and path verify cheyyi → repository connection check cheyyi → application refresh cheyyi → webhook or polling issue unda chudu.
Eng: If Argo CD does not detect the latest Git commit → first verify the commit exists in the configured branch → check repository, branch, and path → verify repository connection → refresh the application → check webhook or polling issues.
Example: git log -1 → latest commit check cheyyi.
Argo CD Application → **Repo URL, Target Revision, Path** verify cheyyi.
argocd app get <app-name> → current revision chudu.
Application refresh cheyyi → latest commit detect ayyinda verify cheyyi.

---

17. Argo CD deployed the application, but the application is not working. What will you check?
Ans: First, I verify whether Argo CD successfully synchronized the resources and whether the Kubernetes resources are Healthy. Then I check the Pods, Services, Ingress, application logs, and configuration.
I also verify whether the correct image, environment variables, ConfigMaps, Secrets, ports, and health probes are being used. After identifying the Kubernetes or application issue, I fix it and verify the application again.
Simple ga gurthupettuko: Argo CD deploy successful ayina app work avvakapothe → first resources Healthy unnaya chudu → Pods and Services check cheyyi → logs and configuration verify cheyyi → image, variables, ConfigMap, Secret and ports check cheyyi → issue fix chesi app verify cheyyi.
Eng: If Argo CD deploys successfully but the application does not work → first verify resource health → check Pods and Services → verify logs and configuration → check image, variables, ConfigMap, Secret, and ports → fix the issue and verify the application.
Example: kubectl get pods -n app → Pods Running and Ready unnaya chudu.
kubectl logs <pod-name> -n app → application error check cheyyi.
kubectl get svc -n app → Service port verify cheyyi.

kubectl describe pod <pod-name> -n app → ConfigMap, Secret and probe issues chudu.
Issue fix chesi → Argo CD refresh → application **Healthy** ayyinda verify cheyyi.

---

18. Argo CD application keeps going back to OutOfSync after every sync. What will you check?
Ans: First, I check the application diff after synchronization to identify which resource or field keeps changing. Then I verify whether some external process, controller, or manual change is modifying the resource.
I also check whether the Kubernetes controller is adding or changing fields that are not present in Git. If the difference is expected, I configure the appropriate Argo CD ignore differences behavior; otherwise, I fix the source of the unwanted change.
Simple ga gurthupettuko: Sync chesina malli OutOfSync ayithe → first repeated difference identify cheyyi → external controller or manual change unda chudu → Kubernetes resource fields change chesthunda check cheyyi → expected difference ayithe ignore configure cheyyi → unexpected ayithe root cause fix cheyyi.
Eng: If the application becomes OutOfSync again after every sync → first identify the repeated difference → check external controllers or manual changes → verify whether Kubernetes is modifying fields → ignore expected differences → fix unexpected changes.
Example: Argo CD **Diff** → same field repeated ga change avutunda chudu.
kubectl get <resource> <name> -o yaml → live value check cheyyi.
Git manifest tho compare cheyyi → external controller change chesthunda identify cheyyi.
Expected change ayithe → Argo CD ignore-differences configure cheyyi → Sync chesi status stable ga unda verify cheyyi.

---

19. Argo CD application is stuck during deletion. What will you check?
Ans: First, I check which resource is preventing the application from being deleted. Then I inspect the resource status, events, and finalizers.
I also check whether the resource has dependencies or whether another controller is preventing its deletion. I remove a finalizer only when I understand why it is blocking deletion and when it is safe to do so.
Simple ga gurthupettuko: Application delete avvakapothe → first ye resource deletion block chesthundo chudu → resource status and events check cheyyi → finalizers verify cheyyi → dependencies check cheyyi → safe ayithe blocking issue fix chesi deletion verify cheyyi.
Eng: If the Argo CD application is stuck during deletion → first identify the resource blocking deletion → check resource status and events → verify finalizers → check dependencies → safely fix the blocking issue and verify deletion.
Example: kubectl get application <app-name> -n argocd -o yaml → finalizers and status chudu.
kubectl get resources -n <namespace> → remaining resources identify cheyyi.
kubectl describe <resource> <name> -n <namespace> → deletion issue chudu.
Finalizer reason understand chesi → safe ga issue fix cheyyi → Application delete ayyinda verify cheyyi.

---

20. Argo CD application is Healthy but users are still getting errors. What will you check?
Ans: First, I understand that Healthy status only indicates that Argo CD sees the Kubernetes resources as healthy; it does not guarantee that the application is functioning correctly for users.
Then I check the application logs, Service, Ingress, DNS, network connectivity, backend dependencies, and application-level errors. I also verify whether the correct version and configuration are deployed. After finding the actual application issue, I fix it and verify the user request flow end to end.
Simple ga gurthupettuko: Argo CD Healthy unna users errors vaste → first Healthy ante Kubernetes resources okay ani artham ani remember cheyyi → app logs chudu → Service and Ingress check cheyyi → DNS and network verify cheyyi → backend dependency and configuration chudu → actual issue fix chesi user request end-to-end verify cheyyi.
Eng: If Argo CD shows Healthy but users still get errors → remember Healthy only confirms resource health → check application logs → verify Service and Ingress → check DNS and network → verify backend dependencies and configuration → fix the actual issue and test the complete user request flow.
Example: Argo CD **Healthy** → still user ki error vaste kubectl logs <pod-name> -n app → application error chudu.
kubectl get svc -n app → Service configuration verify cheyyi.
kubectl get ingress -n app → Ingress hostname and backend check cheyyi.
nslookup app.example.com → DNS resolve avutunda chudu.
Issue fix chesi → browser/API request malli test chesi → application properly work avutunda verify cheyyi.